

Gene Family Classification Report

In this report we assess the predictive performance of two models at classifying a given gene's DNA into the correct gene family. The two models we assessed were a "naive" kmer count logistic regression (KCLR) model and the "refined" Nucleotide Transformer 50M (NT50M) model fine-tuned with IA3.

Exploratory Data Analysis

We were provided a dataset of 4,380 gene DNA sequences. When we look at the distribution of gene lengths we see a heavily left-skewed distribution where the median is around 865. The shortest sequence is 5 nucleotides, with the longest being 18,921 nucleotides. When we look at the distribution of gene families we see a class imbalance, with the smallest family having just over 200 sequences, and the largest having over 1,300 sequences. When we look at the distribution of gene lengths we see that these are very similar between families, this is also true for the distribution of specific nucleotide bases.

Model Choices

For our "naive" baseline model we chose kmer count logistic regressor (KCLR), which first converts a DNA sequence into a histogram of kmer counts, and then uses the histogram embedding to train a logistic regressor. This was chosen as it is a common model used by bioinformaticians.

For our "refined" model we chose the Nucleotide Transformer V2 50M (NT50M) parameter multi-species model. This was chosen as it is a popular choice for DNA classification and we have experience with the architecture. This encoder-only transformer model has already been pre-trained on a large corpus of DNA from multiple species using the Masked Language Modelling training objective. This should mean gene embeddings from the same family should be closer together in embedding space as they should share common motifs. For training we treat the NT50M model as a backbone to generate embeddings and attach a Multi-Perceptron Layer (MPL) classification head. To prevent the model from overfitting we freeze the backbone weights and insert IA3 trainable weights into models key, value, and forward layers to encourage the embedding space to shift towards a representation more aligned with the task. Furthermore, we use weighted cross-entropy loss to address the class imbalance.

Validation Methodology

To validate our models effectively, we split our original dataset into training, validation, and test splits, at a ratio of 8:1:1. We will use the training dataset to tune the model parameters, the validation dataset to tune the model hyper-parameters, and the test dataset as a final generalisability performance measure.

We throw away any genes less than 18 nucleotides as the NT50M model has a resolution of 6 nucleotides, making it difficult to classify anything shorter. We also throw away genes with more than 4092 nucleotides, as training on sequences longer than this is too slow on the

available hardware given the 24 hour time limit. Some sequences are very similar to others, so we need to cluster them to avoid data leakage. To achieve effective splits while maintaining similar gene family distributions, we use MMseqs2 to cluster sequences within a gene family. For each family, we take the biggest clusters first and alternatingly add them to each dataset split.

To assess the performance of our models we use a mix of classification metrics. As there are 7 gene families, we can assess the model on both the individual family's classification performance and the aggregated classification performance across the families. We use accuracy to get a general idea of performance, weighted vs average F1 scores per family to compare precision vs recall and how the class imbalance impacts this, along with multi-class Matthews correlation as a strict measure of performance as it penalises biases strongly.

Results

During training and validation of the naive KCLR model, we found that the model performed best using 8-mer embeddings with around 100 iterations of training. For the NT50M model, there were a lot more hyper-parameters to test and with the limited time available it is unlikely the best hyper-parameters were chosen. The model reached maximum performance on the validation dataset after 3 full training epochs of the data, with a learning rate of 1e-3 and a dropout of 0.1 on the classification head.

Overall, neither model showed a strong classification performance at the gene family task. When comparing the models we find that the KCLR model outperforms the NT50M model on almost every metric. Given that the KCLR model performs best with 8-mers and the NT50M uses 6-mer tokens, it could be that the motifs that correlate most with gene families are 8 nucleotides long, and that the NT50M model misses these due to its resolution constraints. When we look at the per family F1 scores between the models, we see that the KCLR has very low recall of gene family "2", which might imply 8-mers are unsuitable for the motifs found in this family.

Model	Accuracy	Average F1	Weighted F1	Matthews Coef
KCLR	0.56	0.43	0.51	0.43
NT50M	0.48	0.40	0.49	0.37

Table 1. The model classification metrics on the gene family test dataset.

Conclusion

In conclusion, we successfully validated the performance of the "naive" kmer count logistic regression (KCLR) model and the "refined" Nucleotide Transformer 50M (NT50M) model fine-tuned with IA3. Overall, neither of the models showed a strong classification performance, but out of the two the KCLR model outperformed the NT50M model on every metric. Given more time to tune the NT50M model's hyper-parameters the model might have outperformed the KCLR model. However, given the 6 nucleotide resolution constraint, the KCLR model suggests that a single-nucleotide resolution "refined" model might perform better.